Comprehensive Tutorial on Multilayer Perceptron (MLP) for Image Classification

Qumrish Arooj
MS DS
Machine Learning and Neural Networks

***Abstract*—This tutorial presents a detailed study of the Multilayer Perceptron (MLP) for image classification using the MNIST dataset. Multiple experiments are performed by modifying parameters such as network depth, activation functions, optimizers, and dropout. The performance is evaluated using accuracy, training time, confusion matrices, and prediction visualization. The objective of this tutorial is to provide a beginner-friendly understanding of neural networks and their practical implementation.**

I. Introduction

Machine learning is an important area of artificial intelligence that allows systems to learn patterns from data. Among various machine learning models, neural networks are widely used for solving complex problems such as image classification. A Multilayer Perceptron (MLP) is one of the simplest forms of neural networks and serves as a strong foundation for understanding deep learning[1].

In this tutorial, an MLP model is implemented to classify handwritten digits. The main goal is to understand how different parameters affect model performance. Multiple experiments are conducted, and results are analyzed in detail. This tutorial is written in simple language so that beginners can easily understand the concepts.

II. Dataset Description

The MNIST dataset is used in this study. It contains 70,000 grayscale images of handwritten digits from 0 to 9. Each image has a resolution of 28×28 pixels. The dataset is divided into training and testing sets. The training set is used to train the model, while the testing set is used to evaluate performance.

Before training, the data is preprocessed by normalizing pixel values to the range of 0 to 1. This improves training stability. The labels are converted into one-hot encoded vectors so that the model can perform multi-class classification[2].

III. Methodology

The Multilayer Perceptron consists of an input layer, hidden layers, and an output layer. The input layer receives pixel values, while hidden layers extract features. The output layer predicts the digit [3].

The computation inside a neuron is defined as:

$$z = \sum w_i x_i + b \tag{1}$$

$$a = f(z) \tag{2}$$

where w_i represents weights, x_i represents inputs, b is bias, and f is the activation function.

The ReLU activation function is defined as:

$$f(x) = \max(0, x) \tag{3}$$

The Tanh activation function is defined as:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{4}$$

The loss function used is categorical crossentropy:

$$L = -\sum y \log(\hat{y}) \tag{5}$$

IV. Experimental Setup

Different experiments are conducted to analyze the effect of parameters. The following table shows the configurations used in each experiment.

TABLE I
Experiment Parameters

Experiment	Layers	Activation	Optimizer	Dropout
Baseline	[128,64]	ReLU	Adam	No
Deep Network	[256,128,64]	ReLU	Adam	No
Tanh	[128,64]	Tanh	Adam	No
SGD	[128,64]	ReLU	SGD	No
Dropout	[128,64]	ReLU	Adam	Yes

The Adam optimizer is an adaptive learning rate optimization algorithm that combines the advantages of both AdaGrad and RMSProp, making it highly efficient for training deep neural networks [4].

V. Results

TABLE II
Model Comparison

Experiment	Accuracy	Time (sec)
Baseline	0.9709	9.13
Deep Network	0.9770	12.34
Tanh	0.9733	9.62
SGD	0.9459	9.46
Dropout	0.9714	10.33

Dropout is a regularization technique used to reduce overfitting by randomly disabling neurons during training, forcing the model to learn more robust features [5].

VI. Plots and Analysis

A. Accuracy Plot

The accuracy plot shows how the model learns over time. It can be observed that training accuracy increases steadily while validation accuracy stabilizes after a few epochs. This indicates that the model is learning effectively, although slight overfitting may occur in later epochs.

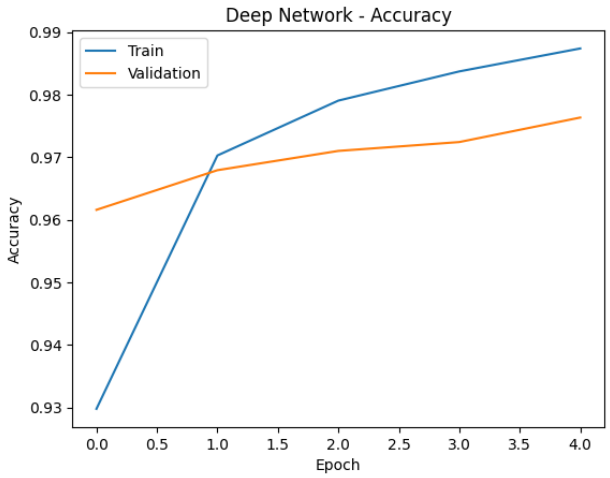

Fig. 1. Training vs Validation Accuracy

B. Loss Plot

The loss plot shows the decrease in error during training. Initially, the loss decreases rapidly, which means the model is learning quickly. Later, the validation loss stabilizes, indicating that further training does not significantly improve performance.

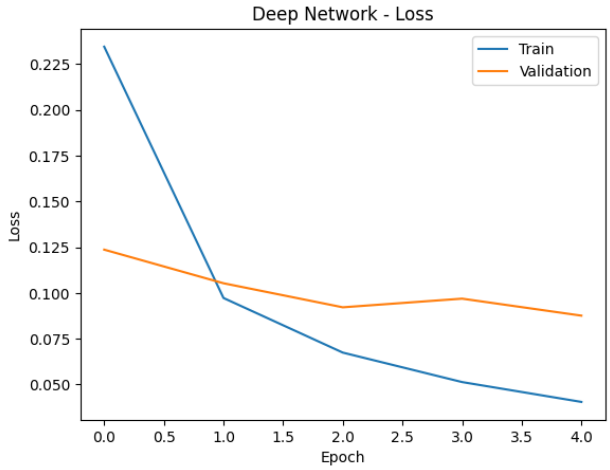

Fig. 2. Training vs Validation Loss

C. Confusion Matrix

The confusion matrix provides a detailed view of model predictions. Most values are concentrated along the diagonal, indicating correct predictions. Some misclassifications occur between similar digits, such as 3 and 5.

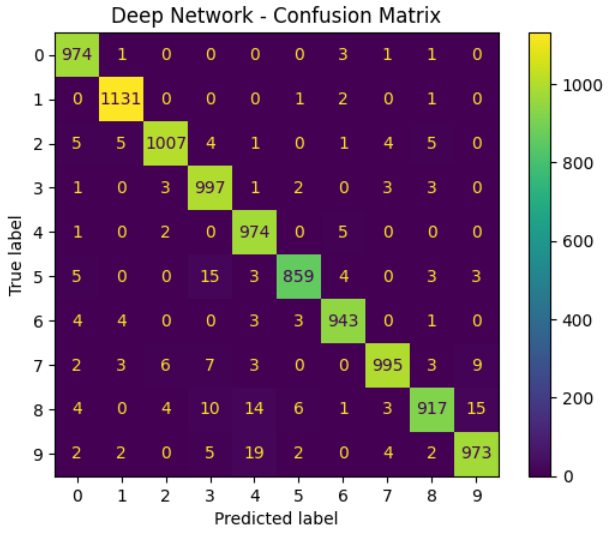

Fig. 3. Confusion Matrix

D. Model Comparison Plot

This plot compares the accuracy of different models. The deep network achieves the highest accuracy, while the SGD optimizer performs the worst. This highlights the importance of choosing the right model configuration.

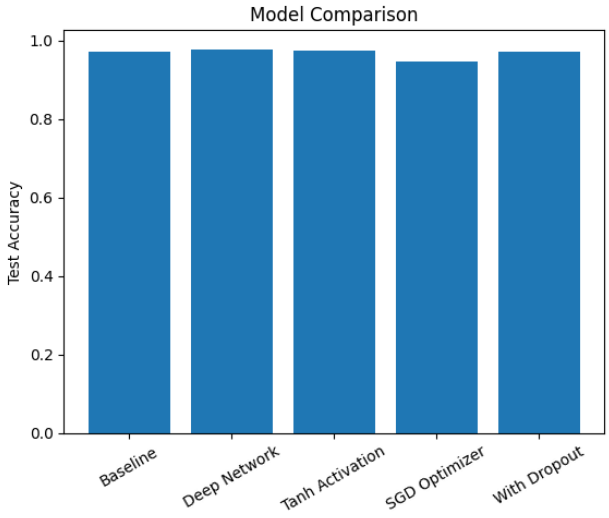

Fig. 4. Accuracy Comparison

E. Prediction Results

The prediction plot for the Deep Network visually demonstrates how well the model classifies handwritten digits from the MNIST test set. In this plot, a selection of test images is shown alongside the model's predicted labels. Most of the predictions match the true labels, confirming that the network

has successfully learned to recognize the patterns of different digits. A few errors may occur between visually similar digits, such as 3 and 5 or 4 and 9, which is a common challenge in handwritten digit recognition. This visualization helps to validate the model's performance beyond numerical metrics, providing an intuitive understanding of how the neural network interprets and predicts each input image.

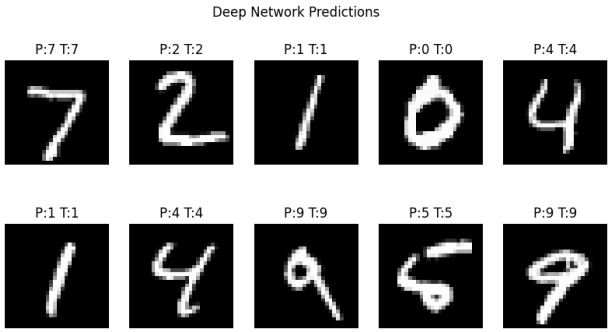

Fig. 5. Sample predictions of the Deep Network on the MNIST test dataset. Each image shows the true label and the predicted label by the model. Most predictions match the true labels, with very few misclassifications.

VII. Discussion

The results show that increasing the number of layers improves model performance but also increases training time. The Adam optimizer performs better than SGD due to its adaptive learning rate. ReLU activation is more effective than Tanh because it avoids gradient issues. Dropout helps reduce overfitting and improves generalization.

VIII. Code Availability

The complete implementation of this project, including all experiments, plots, and source code, is publicly available at:
GitHub Repository

IX. Conclusion

This tutorial demonstrates how an MLP can be used for image classification. By experimenting with different parameters, it becomes clear that model performance depends heavily on architecture and training choices. The results provide valuable insights into neural network design and optimization.

X. References

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org
[2] Y. LeCun and C. Cortes, "MNIST handwritten digit database," 2010. [Online]. Available: http://yann.lecun.com/exdb/mnist/
[3] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/
[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.